

# RED-BLACK TREE

A Red-Black Tree is a self-balancing binary search tree with a height limit of  $O(\log N)$ , enabling efficient search, insertion, and deletion operations in  $O(\log N)$  time, unlike standard binary search trees which can take  $O(N)$  time.

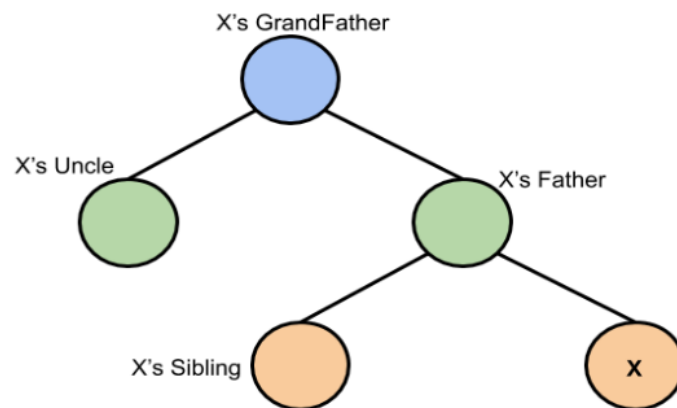
- Each node has an additional attribute: a color, which can be either red or black.
- These colors are used to maintain balance during insertions and deletions, ensuring efficient data retrieval and manipulation.

## Properties of Red-Black Trees

A Red-Black Tree have the following properties:

1. **Node Color:** Each node is either red or black.
2. **Root Property:** The root of the tree is always black.
3. **Red Node Property:** Red nodes cannot have red children (Red nodes cannot be adjacent).
4. **Black Node Property:** Every path from a node to its descendant leaves must have the same number of black nodes.
5. **Leaf Property:** All leaves (NIL nodes) are black.

The representation we will be working with is:

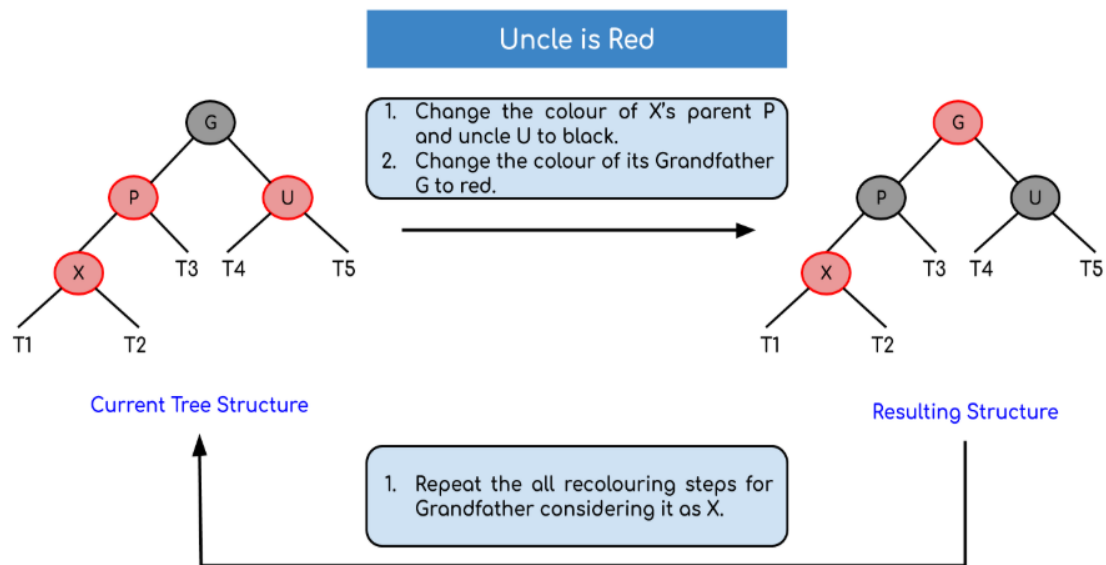


This representation is based on X

## Logic:

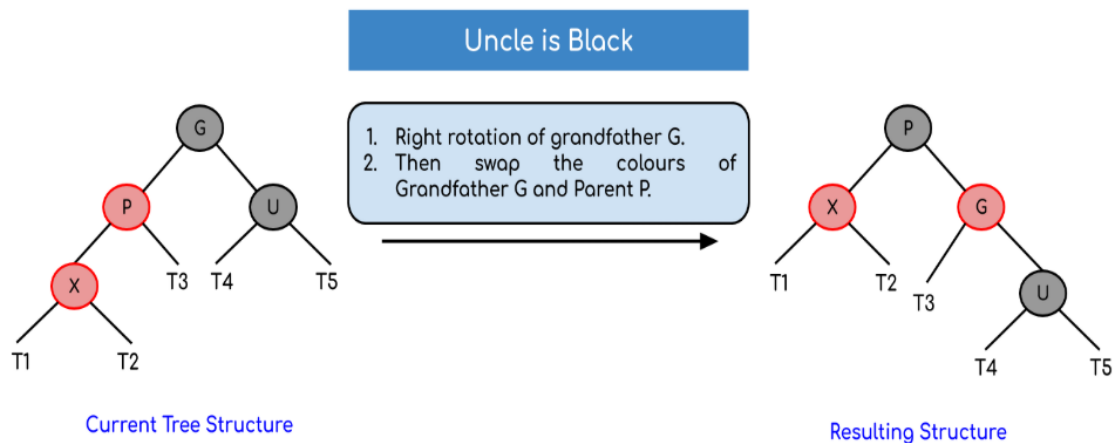
First, you have to insert the node similarly to that in a binary tree and assign a red colour to it. Now, if the node is a root node then change its colour to black, but if it is not then check the colour

of the parent node. If its colour is black then don't change the colour but if it is not i.e. it is red then check the colour of the node's uncle. If the node's uncle has a red colour then change the colour of the node's parent and uncle to black and that of grandfather to red colour and repeat the same process for him (i.e. grandfather). If grandfather is root then don't change grandfather to red colour.

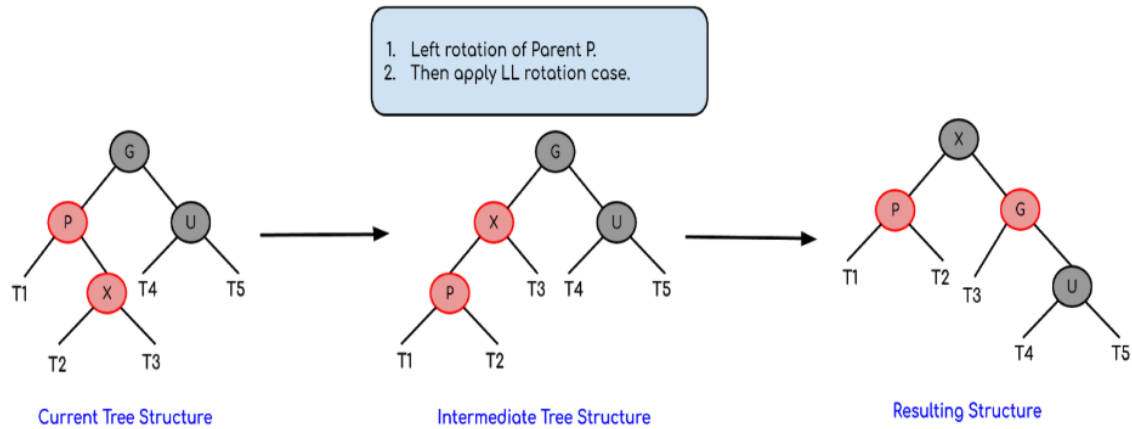


But, if the node's uncle has black colour then there are 4 possible cases:

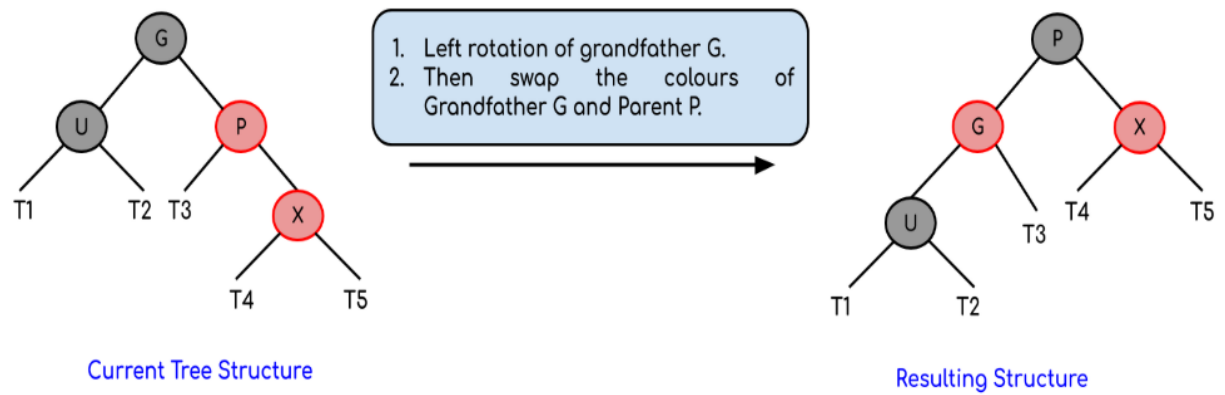
### Left Left Case (LL rotation):



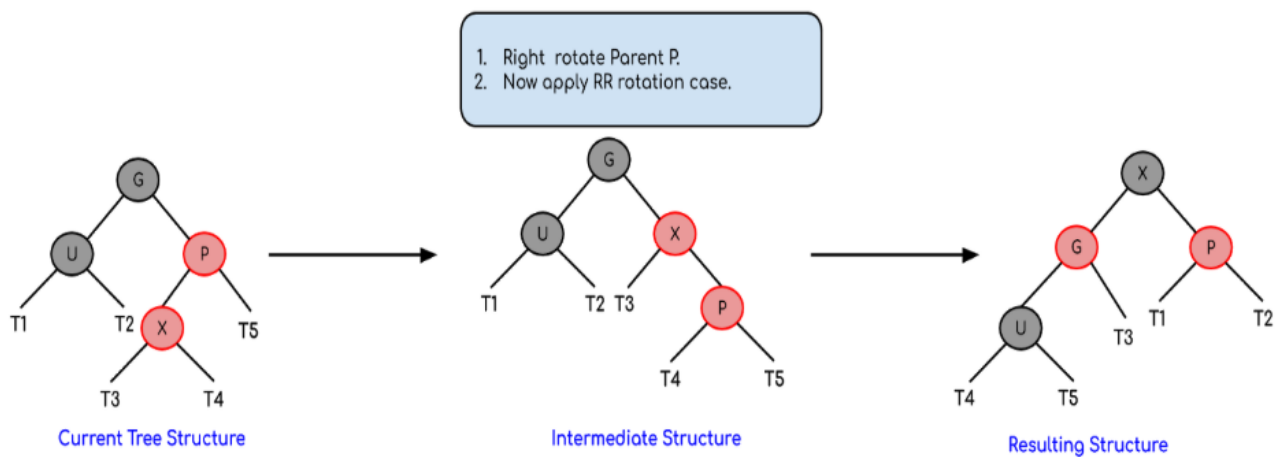
### Left Right Case (LR rotation):



### Right Right Case (RR rotation):



### Right Left Case (RL rotation):



## Operations on Red-Black Tree:

1. Insertion
2. Searching
3. Deletion:
4. join
- 5 split

## 1. Insertion:

A Red–Black Tree is a self-balancing Binary Search Tree where each node has a color:

-  Red
-  Black

It guarantees  $O(\log n)$  search, insert, and delete.

### Core Rules (Must Remember)

1. Every node is Red or Black
2. Root is Black
3. New node is always Red
4. No two consecutive Red nodes
5. Every path from node to NULL has same black count

## Insertion Algorithm

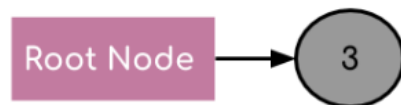
### INSERT(x):

1. Insert x like BST
2. Color x as RED
3. While parent(x) is RED:
  - a) If uncle is RED  $\rightarrow$  Recolor
  - b) Else  $\rightarrow$  Rotate + Recolor
4. Color root BLACK

## Example: Creating a red-black tree with elements 3, 21, 32 and 15 in an empty tree.

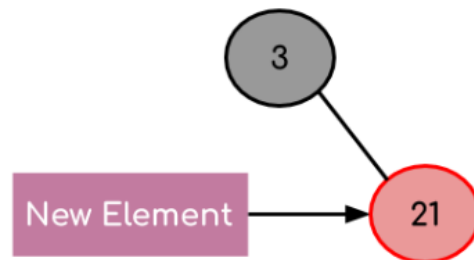
**Solution:**

**Step 1:** Inserting element 3 inside the tree.



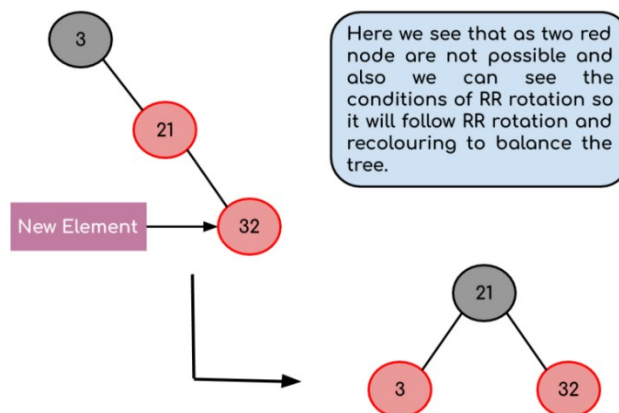
When the first element is inserted it is inserted as a root node and as root node has black colour so it acquires the colour black.

**Step 2:** Inserting element 21 inside the tree.



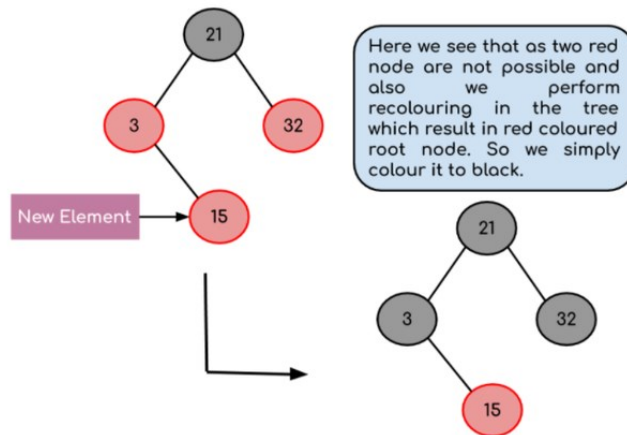
The new element is always inserted with a red colour and as  $21 > 3$  so it becomes the part of the right subtree of the root node.

**Step 3:** Inserting element 32 inside the tree.

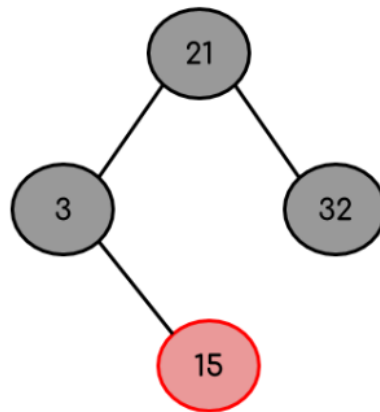


Now, as we insert 32 we see there is a red father-child pair which violates the Red-Black tree rule so we have to rotate it. Moreover, we see the conditions of RR rotation (considering the null node of the root node as black) so after rotation as the root node can't be Red so we have to perform recolouring in the tree resulting in the tree shown above.

**Step 4:** Inserting element 15 inside the tree.



**Final Tree Structure:**



**Example :2 Create Red-Black Tree of the following Elements {1,2,3,4,5,6,7,8}**

**Solution:**

**Step 1: Insert 1**

- Tree empty  $\rightarrow$  root
- Root must be black

(1 ●)

**Step 2: Insert 2**

- $2 > 1 \rightarrow$  right
- Insert as red

(1 ●)

(2 ●)

✓ No violation

### Step 3: Insert 3

- $3 > 1 \rightarrow$  right
- $3 > 2 \rightarrow$  right
- Red-Red violation (2,3)

→ Right-Right case → Left Rotation

(2 ●)

(1 ●) (3 ●)

### Step 4: Insert 4

- Goes to right of 3
- Parent & uncle both red

→ Recolor

(2 ●)

(1 ●) (3 ●)  
(4 ●)

### Step 5: Insert 5

- Goes to right of 4
- Red-Red violation

→ Left Rotation at 3

(2 ●)

(1 ●) (4 ●)  
(3 ●) (5 ●)

### Step 6: Insert 6

- Goes to right of 5
- Uncle is red

→ Recolor

(2 ●)

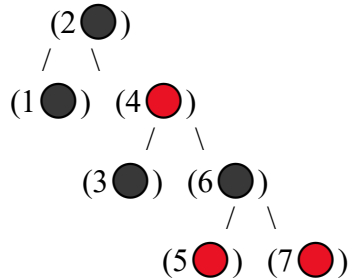
(1 ●) (4 ●)  
(3 ●) (5 ●)

(6●)

**Step 7: Insert 7**

- Goes to right of 6
- Red-Red violation

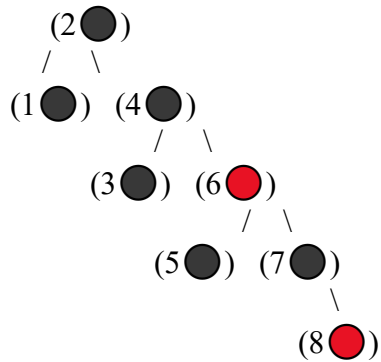
**→ Left Rotation at 5**



**Step 8: Insert 8**

- Goes to right of 7
- Uncle is red

**→ Recolor**



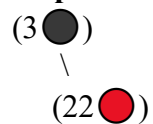
**Example :2 Create Red-Black Tree of the following Elements {3, 22, 32, 15, 8}**

Solution:

**Step 1: Insert 3**

(3●)

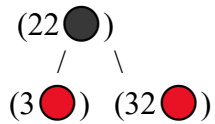
**Step 2: Insert 22**



**Step 3: Insert 32**

- Red-Red violation

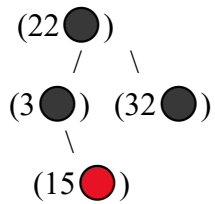
→ **Left Rotation**



**Step 4: Insert 15**

- Goes to right of 3
- Uncle is red

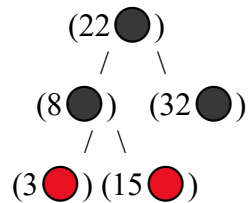
→ **Recolor**



**Step 5: Insert 8**

- Goes left of 15
- Left-Right case

→ **Left Rotation + Right Rotation**



## 2 Delete Operation:

Deleting a node may **break black-height balance**.  
So after deletion, we must **fix “double black” problems**.

### Algorithm

DELETE(x):

1. Find node x (BST delete)
2. If x has 2 children → replace with inorder successor
3. Remove node
4. If removed node is RED → done
5. If removed node is BLACK → fix DOUBLE BLACK
6. Use:
  - Recolor
  - Rotation
7. Ensure root is BLACK

### Double Black Fix Cases :

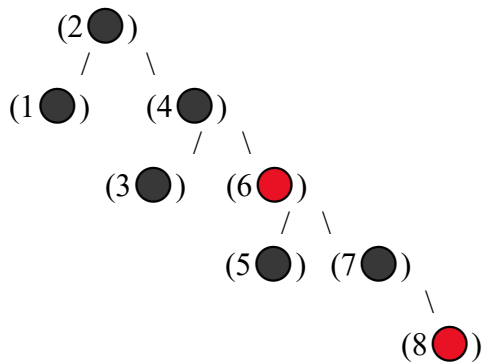
| Case   | Condition                          | Fix              |
|--------|------------------------------------|------------------|
| Case 1 | Sibling RED                        | Rotate + recolor |
| Case 2 | Sibling BLACK, both children BLACK | Recolor sibling  |
| Case 3 | Sibling BLACK, near child RED      | Rotate sibling   |
| Case 4 | Sibling BLACK, far child RED       | Rotate parent    |

### Double Black Algorithm:

FIX-DOUBLE-BLACK(T, x):

1. If x is root:  
    Remove double black.  
    Return.
2. Let s = sibling of x  
    Let p = parent of x
3. If s is RED:           // Case 1  
    Swap colors of s and p  
    Rotate at p toward x  
    Call FIX-DOUBLE-BLACK(T, x)
4. Else if s is BLACK and  
    both children of s are BLACK or NULL:   // Case 2  
    Color s RED  
    If p is RED:  
        Color p BLACK  
        Remove double black  
    Else:  
        Mark p as DOUBLE BLACK  
        Call FIX-DOUBLE-BLACK(T, p)
5. Else if s is BLACK and  
    near child of s is RED and  
    far child of s is BLACK:                // Case 3  
    Swap colors of s and near child  
    Rotate at s away from x  
    Call FIX-DOUBLE-BLACK(T, x)
6. Else if s is BLACK and  
    far child of s is RED:                // Case 4  
    Set s.color = p.color  
    Color p BLACK  
    Color far child BLACK  
    Rotate at p toward x  
    Remove double black

**EXAMPLE :**DELETE node 6 and node 4 of the following **GIVEN RED-BLACK TREE**



**Solution:**

**DELETE NODE = 6**

**Step 1: Find Node 6**

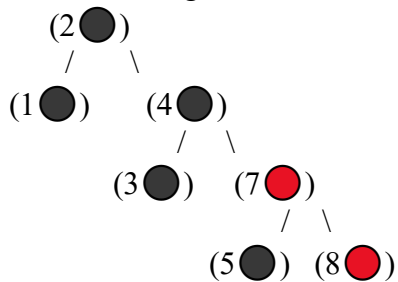
- Node 6 is **RED**
- Has **two children**

**Step 2: Find Inorder Successor**

- Successor of 6 = 7

**Step 3: Replace 6 with 7**

- Copy value 7 into node 6
- Delete original node 7



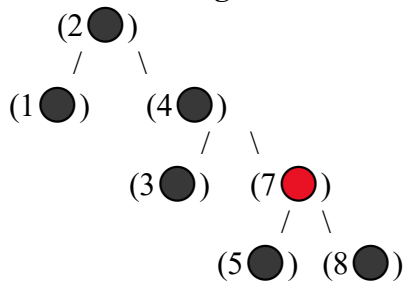
**Step 4: Delete Original 7**

- Node 7 was **BLACK**
- Child = **8 (RED)**

→ Replace 7 with 8

→ Color 8 **BLACK**

### Tree After Deleting 6



✓ No double black

✓ Tree balanced

### DELETE NODE = 4

#### Step 1: Find Node 4

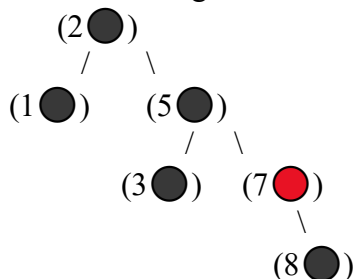
- Node 4 is **BLACK**
- Has **two children**

#### Step 2: Find Inorder Successor

- Successor of 4 = **5**

#### Step 3: Replace 4 with 5

- Copy value **5** into node 4
- Delete original node 5



#### Step 4: Delete Original 5

- Node 5 was **BLACK**
- Leaf node

→ Causes **DOUBLE BLACK** at its position

#### Step 5: Fix Double Black

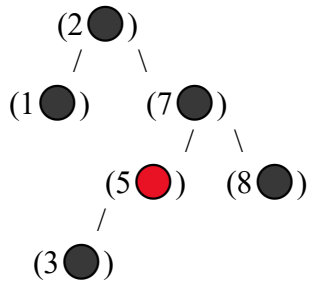
- Sibling = 7 (**RED**)

→ **Case 1: Red Sibling**

**Fix:**

- Recolor sibling & parent
- Rotate **LEFT** at parent (**5**)

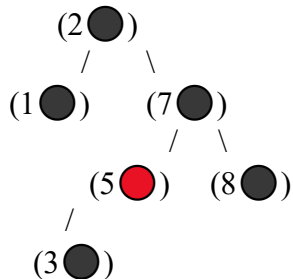
After fix:



#### Step 6: Recolor

- Remove extra black
- Ensure root black

#### FINAL TREE AFTER DELETING 6 AND 4



#### Example 3 :

Construct a **Red-Black Tree** by inserting the elements **10, 15, 9, 12, 13, 79, 45, 36, and 22** in the given order, showing the tree after each insertion with proper node coloring and any rotations or recoloring performed; after obtaining the final tree, further perform the operations

- insert 5**
- insert 3**
- delete 79** and
- delete 10** .

#### Solution:

##### Insert Elements

Sequence: 10, 15, 9, 12, 13, 79, 45, 36, 22

##### INSERTION PHASE

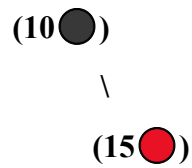
##### Step 1: Insert 10

**Tree      Explanation**

(10 ●) Tree empty → insert as root → root must be black

##### Step 2: Insert 15

**Tree**

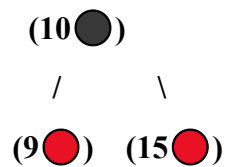


**Explanation**

Insert as BST → parent black → no fix

**Step 3: Insert 9**

**Tree**

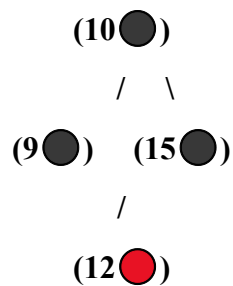


**Explanation**

Parent black → valid

**Step 4: Insert 12**

**Tree**

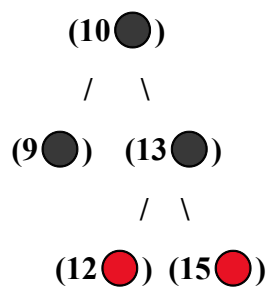


**Explanation**

Red parent & red uncle → recolor

**Step 5: Insert 13**

**Tree**



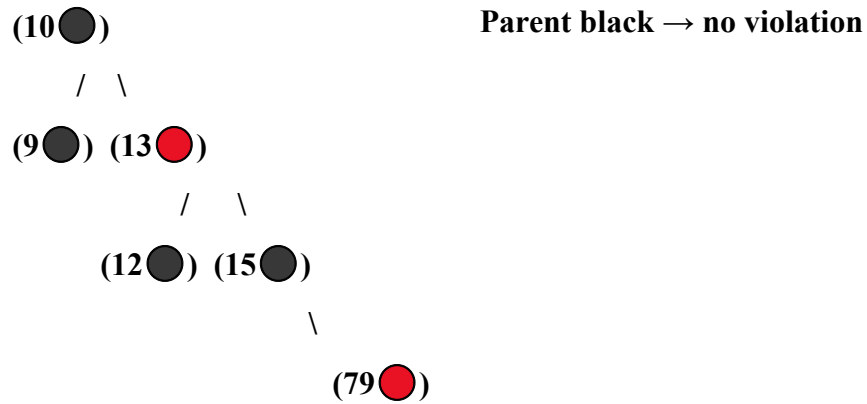
**Explanation**

LR case → rotate + recolor

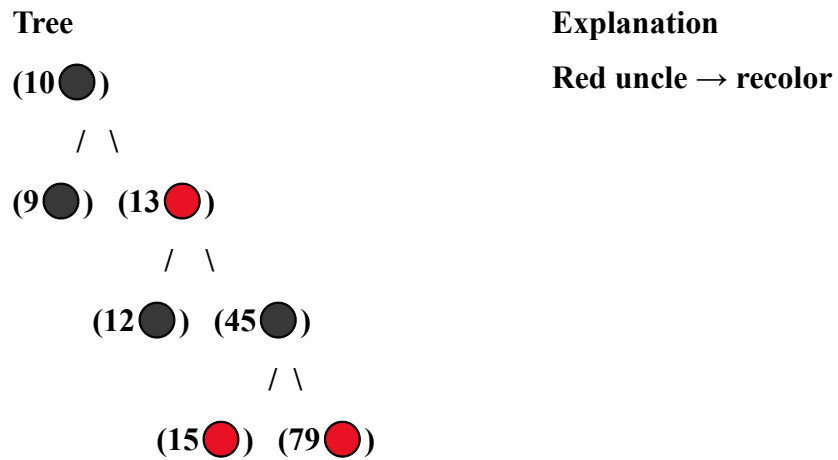
**Step 6: Insert 79**

**Tree**

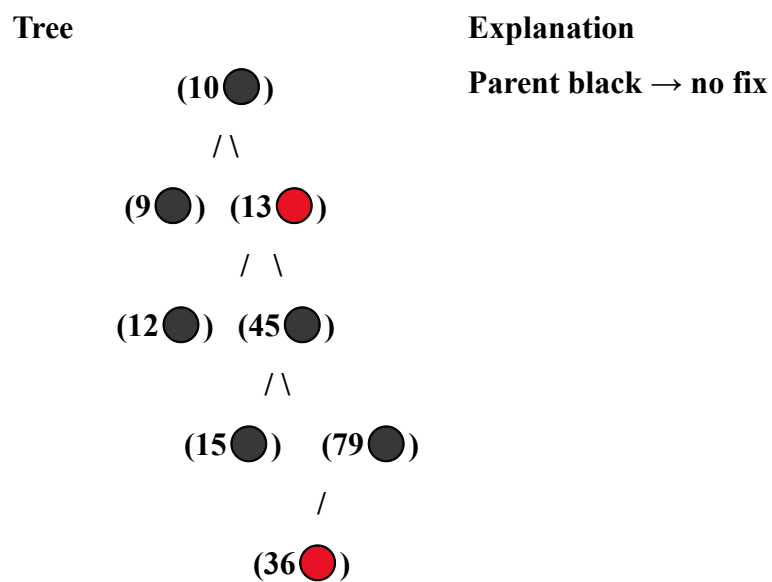
**Explanation**



### Step 7: Insert 45

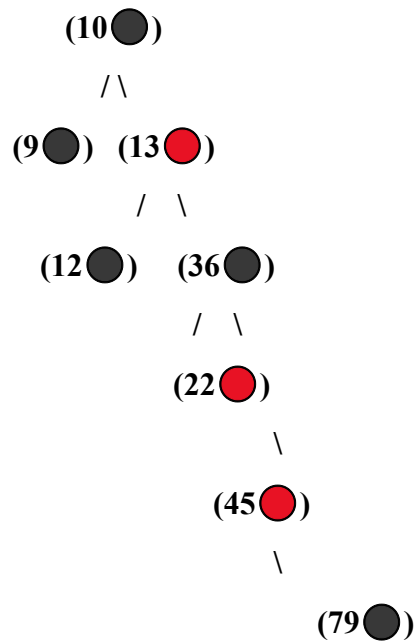


### Step 8: Insert 36



### Step 9: Insert 22

Tree

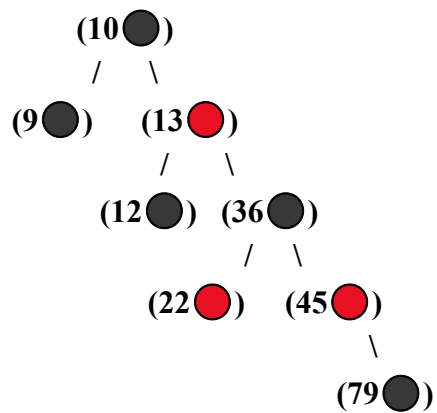


Explanation

Rotation + recolor

i) insert 5

GIVEN RED-BLACK TREE (START)



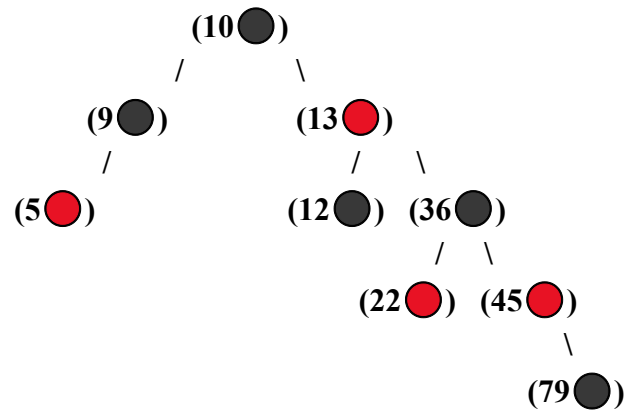
All Red-Black properties are satisfied.

INSERT 5

Step 1: BST Insertion

- $5 < 10 \rightarrow \text{left}$

- $5 < 9 \rightarrow$  left
- Insert as RED



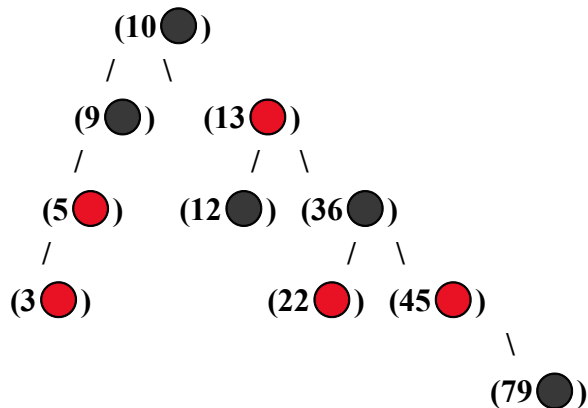
Check violation

- Parent of 5 = 9 (BLACK)
- ✓ No red-red violation  
 → No fix needed

## ii) INSERT 3

Step 2: BST Insertion

- $3 < 10 \rightarrow$  left
- $3 < 9 \rightarrow$  left
- $3 < 5 \rightarrow$  left
- Insert as RED



✗ Violation Detected

- Node 3
- Parent 5
- Red-Red violation

Identify relatives

- Parent = 5

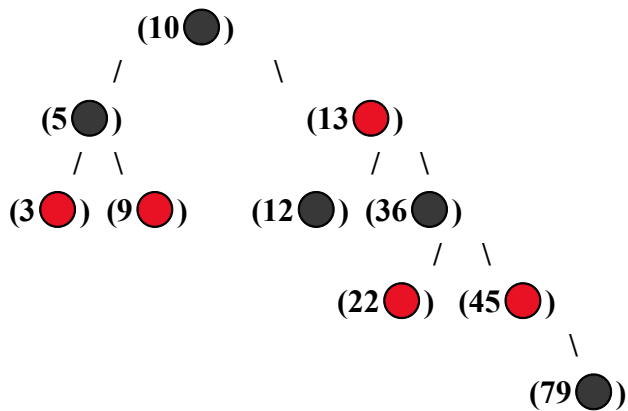
- Uncle = NULL (BLACK)
- Case = Left-Left (LL)

### Step 3: Fix Violation (LL Case)

#### Action

- Right Rotate at 9
- Recolor:
  - 5 → BLACK
  - 9 → RED

#### Tree After Rotation + Recolor

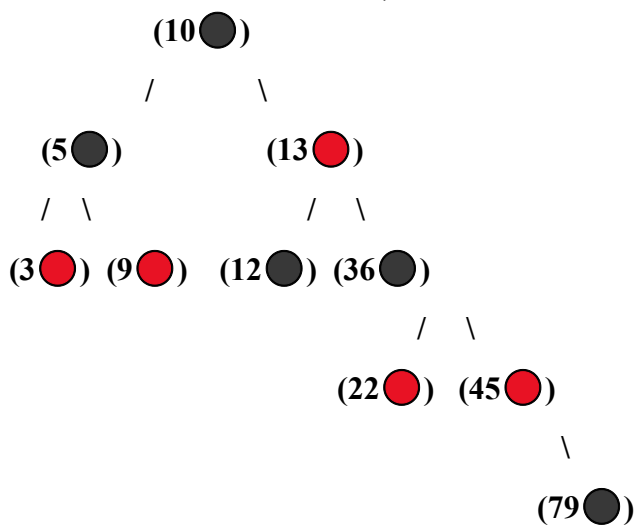


#### Step 4: Root Check

- Root is BLACK
- No red-red
- Black height balanced

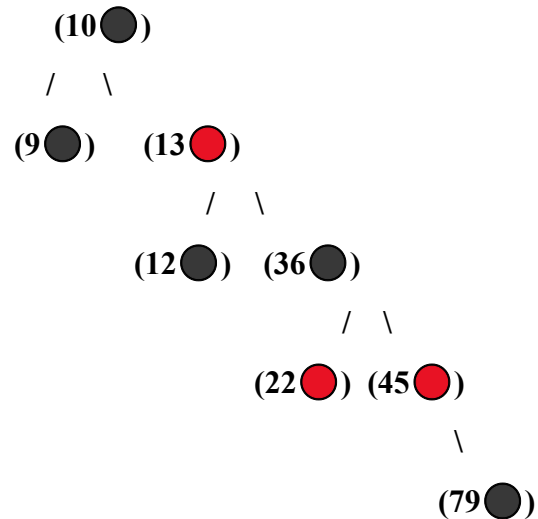
Tree is valid

#### FINAL RED-BLACK TREE (AFTER INSERT 5 AND 3)



### iii Delete 79

#### Initial Red-Black Tree



#### Solution:

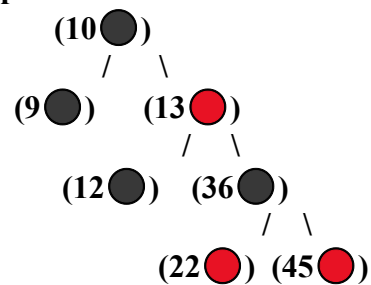
##### DELETE 79

##### Step 1: Locate node 79

- Node 79 (Black)
- It is a leaf node
- Color: BLACK

Deleting a black leaf → may cause double black

##### Step 2: Remove node 79



- A double black appears at the right child of 45

##### Step 3: Fix Double Black

Parent: 45 (Red)

Sibling: NIL (black)

Case:

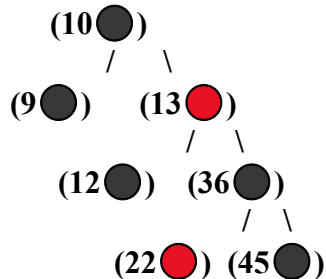
- Parent is RED

- Sibling is BLACK

Fix rule

- Recolor parent → BLACK
- Remove double black

Step 4: Tree after fixing



Red-Black Tree properties restored

#### Iv DELETE 10

Step 1: Locate node 10

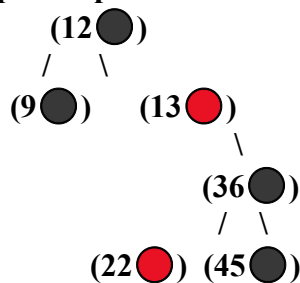
- Node 10 (black)
- Has two children

Use in-order successor

Step 2: Find in-order successor

- Minimum in right subtree → 12 (black)

Step 3: Replace 10 with 12



Now we must delete original 12 (black) from its old position

Step 4: Delete original 12 (black)

- 12 (black) was a black leaf
- Causes double black

Step 5: Fix Double Black

Parent: 13 (red)

Sibling: 36 (black)

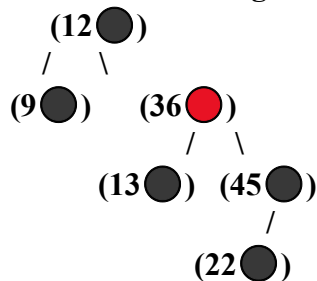
Sibling has red child (22 (red))

**Case: Black sibling with red child**

**Step 6: Apply rotations & recoloring**

- **Left rotate at 13**
- **Recolor:**
  - 36 → takes parent color
  - 13 → BLACK
  - 22 → BLACK

**Final Tree after deleting 10**



### 3 Join Operation:

**JOIN( $T_1$ ,  $x$ ,  $T_2$ )** merges two Red-Black Trees such that:

All keys in  $T_1 < x <$  All keys in  $T_2$

Result is **one valid Red-Black Tree**.

**JOIN Prerequisites (Important)**

- $T_1$  and  $T_2$  are **valid RBTs**
- Every key in  $T_1 < x$
- Every key in  $T_2 > x$

**JOIN Algorithm:**

JOIN( $T_1$ ,  $x$ ,  $T_2$ ):

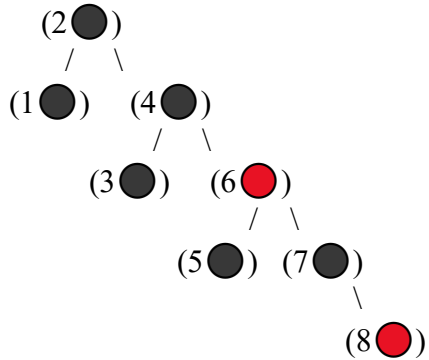
1. Compare black-heights of  $T_1$  and  $T_2$
2. Move down the taller tree until black-heights match
3. Insert  $x$  between the two trees
4. Color  $x$  RED

5. Fix Red–Black violations using rotations & recoloring

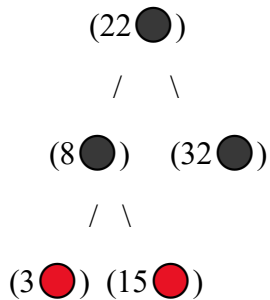
6. Color root BLACK

### Given Trees

Tree  $T_1$  (Set 1: 1–8)



Tree  $T_2$  (Set 2: 3,22,32,15,8)



### Choose JOIN Key

We need a key **greater than all in  $T_1$**  and **less than all in  $T_2$** .

- $\text{Max}(T_1) = 8$
- $\text{Min}(T_2) = 8$

✗ Conflict → same value

So we choose a **new separating key**

Let's take:

$$x = 10$$

$$\checkmark 8 < 10 < 15$$

### Step 1: Compare Black Heights

- Black-height( $T_1$ ) = 3
- Black-height( $T_2$ ) = 2

→  $T_1$  is taller

### Step 2: Move Down $T_1$ (Right Side)

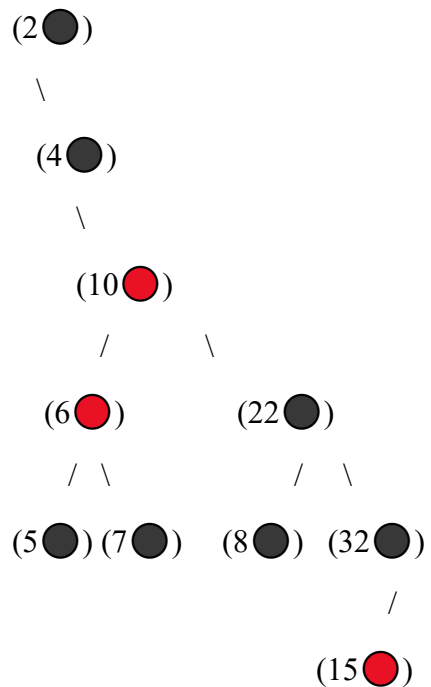
We move down **right spine** of  $T_1$  until black height = 2.

We stop at node:

(6●)

### ⊕ Step 3: Insert JOIN Node $x = 10$

- Make **10 RED**
- Attach:
  - Left child → subtree rooted at (6)
  - Right child →  $T_2$  root (22)



#### ✗ Step 4: Fix Violations

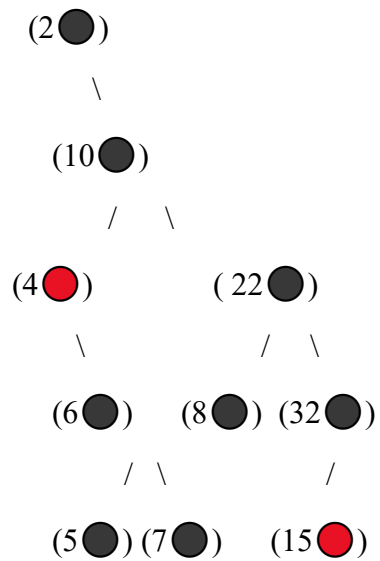
Problem:

- 10 (red) and 6 (red) → Red-Red violation
- Uncle is BLACK

→ Right-Right case

#### 🔄 Step 5: Left Rotation at 4

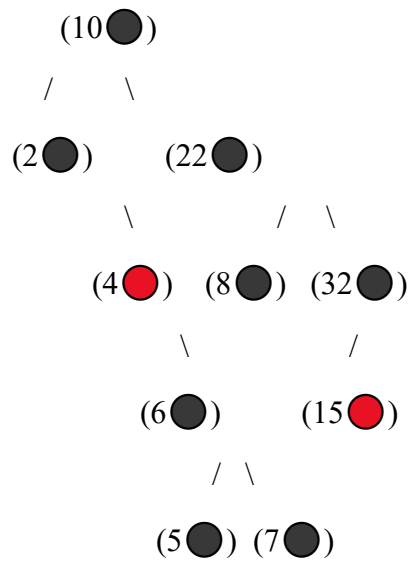
After rotation + recolor:



#### Step 6: Final Recolor

- Root already black
- No red-red
- Black height balanced

## FINAL JOINED RED-BLACK TREE



## 4 Split Operation

The **split operation** divides **one Red-Black Tree T** into **two Red-Black Trees**:

- $T_1 \rightarrow$  all keys  $< x$
- $T_2 \rightarrow$  all keys  $> x$

The key **x** is **removed** (or isolated) from the tree.

Used in **join-split based balanced trees**, range queries, and advanced data structures.

### Idea Behind Split

1. Find node  $x$
2. Break the tree at  $x$
3. Fix Red-Black properties separately in both resulting trees

### Split Operation Steps

Let:

- $L = x.\text{left subtree}$
- $R = x.\text{right subtree}$

**Steps:**

1. Search for key  $x$  like BST search
2. Detach  $x$
3. Treat left subtree as  $T_1$
4. Treat right subtree as  $T_2$
5. While moving up ancestors:
  - Reattach subtrees using **join**
  - Maintain black height
6. Fix coloring and rotations if needed

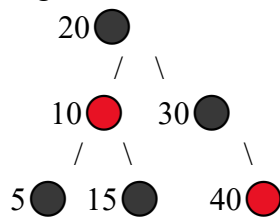
## Split Algorithm

RB-SPLIT( $T, x$ ):

1. Search node  $x$  in  $T$ .
2. Let  $T_1$  = left subtree of  $x$ .
3. Let  $T_2$  = right subtree of  $x$ .
4. Remove  $x$  from  $T$ .
5. For each ancestor  $p$  of  $x$  (moving upward):
  - if  $x$  was in left subtree of  $p$ :
    - $T_2 = \text{RB-JOIN}(p, p.\text{right}, T_2)$
  - else:
    - $T_1 = \text{RB-JOIN}(T_1, p.\text{left}, p)$
6. Color roots of  $T_1$  and  $T_2$  BLACK.
7. Return ( $T_1, T_2$ ).

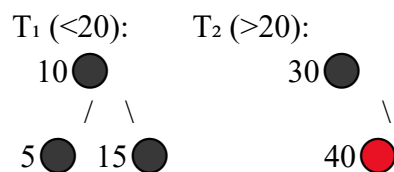
## Example

Original Tree:



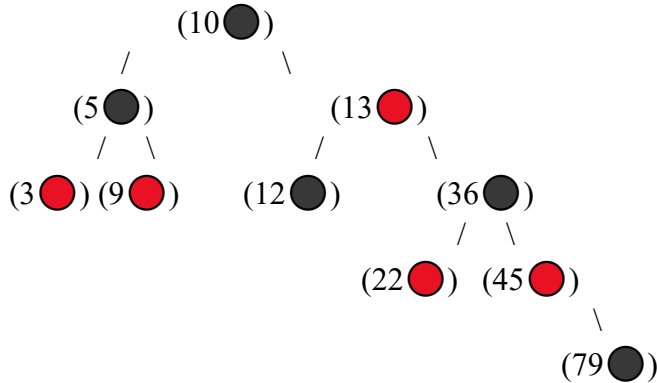
Split at  $x = 20$

Result:



Split operation divides a Red–Black Tree into two balanced Red–Black Trees based on a key while preserving all Red–Black properties using join, recoloring, and rotations.

**Example 2:** split the given Red–Black Tree at key = 12 of the following red black tree



**Solution:**

**Goal of SPLIT at 12**

- $T_1 \rightarrow$  all keys  $< 12$
- $T_2 \rightarrow$  all keys  $> 12$
- Node 12 is removed
- Both resulting trees must remain **valid Red–Black Trees**

**Step 1: Locate Split Node (12)**

- Node 12 (black) is found as the **left child of 13 (red)**
- Detach node 12

**Step 2: Initial Separation**

- **Left subtree of 12**  $\rightarrow T_1$
- **Right subtree of 12**  $\rightarrow T_2$

Since 12 has no children, both start empty.

**Step 3: Move Up and Rebuild Using Join**

**Ancestor: 13 (red)**

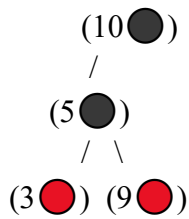
- 12 was **left child**
- Join 13 and its **right subtree** into  $T_2$

**Ancestor: 10 (black)**

- 12 lies in **right subtree**
- Join 10 and its **left subtree** into  $T_1$

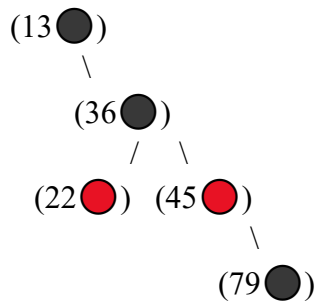
## Final Trees After Split

### Tree $T_1$ (keys < 12)



- ✓ Valid Red–Black Tree
- ✓ Equal black height
- ✓ No red–red violation

### Tree $T_2$ (keys > 12)



- ✓ Root recolored BLACK
- ✓ Red–Black properties preserve
  - Split key **12 is removed**
  - Left side forms  $T_1$  (<12)
  - Right side forms  $T_2$  (>12)
  - Trees rebuilt using **join**
  - Final roots colored **BLACK**

Splitting a Red–Black Tree at key 12 produces two balanced Red–Black Trees: one containing all keys less than 12 and the other containing all keys greater than 12, while maintaining Red–Black properties using join, recoloring, and rotations.

## 5 Search:

Searching in a **Red–Black Tree** is **exactly the same as in a Binary Search Tree (BST)**.

- **Node colors do NOT affect searching.**
- Time complexity is  **$O(\log n)$**  because the tree remains balanced.

### Search Rules (BST Property)

At any node with key  $k$ :

- If  $x == k \rightarrow$  **element found**
- If  $x < k \rightarrow$  go to **left subtree**
- If  $x > k \rightarrow$  go to **right subtree**

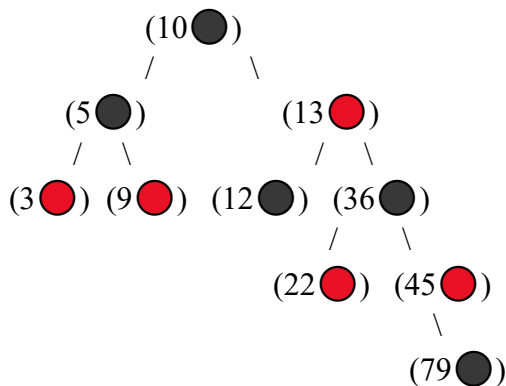
## Algorithm

RB-SEARCH( $root, x$ ):

1. If  $root == NULL \rightarrow$  return NOT FOUND
2. If  $x == root.key \rightarrow$  return FOUND
3. If  $x < root.key \rightarrow$  search( $root.left, x$ )
4. Else  $\rightarrow$  search( $root.right, x$ )

## Example :

Given Red–Black Tree



Implementing Search for Element 22

### Step-by-Step Search Path

| Step | Current Node | Comparison | Action       |
|------|--------------|------------|--------------|
| 1    | 10           | $22 > 10$  | Go right     |
| 2    | 13           | $22 > 13$  | Go right     |
| 3    | 36           | $22 < 36$  | Go left      |
| 4    | 22           | $22 == 22$ | <b>Found</b> |

**Search Path (Highlighted)**

10 → 13 → 36 → 22

- Search successful
- Node 22 ● found
- No rotations or recoloring required

**Note:** hands on practice link <https://ds2-iiith.vlabs.ac.in/exp/red-black-tree/red-black-tree-operations/redblack.html>